

RAPPORT D'ARCHITECTURE LOGICIELLE

Évolution Dynamique du BPMN : Impact Frontend & Gestion du Cycle de Vie des Données

Projet : IncidentFlow V2
Auteur : Anas Benhammou
Axe : BPMN Agility & Instance Migration

1. Modification du Modèle XML : Comportement du Moteur Flowable

Lorsque le schéma XML BPMN 2.0 est modifié (soit par renommage d'un statut, soit par ajout d'une nouvelle tâche utilisateur <userTask>) et réinjecté dans l'application, le moteur d'exécution réagit de façon **strictement dynamique** :

- **Incrémentation de Version (Versioning)** : Flowable ne remplace pas l'ancienne définition ; il crée une nouvelle version (version_ = 2) dans la table flowable.act_re_procdef.
- **Propagation Automatique du Statut** : Dans le service applicatif (IncidentService.java), la méthode de transition interroge la tâche active retournée par le moteur :

```
// Récupération dynamique du nom configuré dans le XML
Task nextTask = taskService.createTaskQuery().processInstanceId(processInstanceId).single
incident.setStatus(nextTask.getName()); // Ex: "En attente validation", "En cours"
```

- **Persistance Métier Immédiate** : Le champ status de la table incidentflow.incidents prend instantanément la valeur littérale déclarée dans le nouvel élément BPMN.

2. Impact sur l'Interface Utilisateur (Tableau Kanban React)

Bien que le moteur backend absorbe nativement toute modification de workflow, l'interface graphique utilisateur (IHM) présente des comportements contrastés selon le degré de couplage :

Type d'Évolution XML	Comportement Moteur Flowable	Comportement Actuel du Kanban
Renommage d'un état existant (ex: "En cours" → "En investigation")	Parfaitement supporté. La nouvelle tâche porte le libellé mis à jour.	Si le libellé ne correspond plus aux 5 colonnes par défaut, la carte est déroutée vers la colonne de secours Autres.
Ajout d'un nouvel état (ex: "En attente client")	Création de la nouvelle étape et routage des tokens d'exécution selon les <sequenceFlow>.	L'incident est persisté avec succès mais reste confiné dans la colonne Autres tant que la colonne n'est pas instanciée graphiquement.

3. Piste d'Amélioration : Le Kanban 100% Dynamique

Pour éliminer tout couplage statique, l'amélioration recommandée consiste à transformer le composant Kanban en un **tableau entièrement piloté par le BPMN (Data-Driven Kanban)** :

Au lieu d'utiliser un tableau de colonnes figé (DEFAULT_COLUMNS), le Frontend interroge au montage un endpoint dédié (ex: GET /api/workflows/active/states) qui extrait la liste ordonnée des <userTask> directement depuis le XML Flowable. Chaque tâche génère automatiquement sa colonne, sa couleur thématique et sa limite WIP (Work In Progress).

4. Risques & Effets de Bord : Que Faire des Incidents Déjà Créés ?

Le déploiement d'un Kanban dynamique introduit un défi classique d'ingénierie logicielle : **la gestion des données résiduelles créées sous l'ancien schéma.**

⚠ Risque 1 : Les "Tickets Orphelins"

Si un état est supprimé ou renommé (ex: "Assigné" devient "En prise en charge"), les anciens tickets ouverts restent enregistrés avec le statut "Assigné". Si la colonne disparaît du Kanban dynamique, ces tickets deviennent invisibles ou inaccessibles.

⚠ Risque 2 : Désynchronisation BPMN

L'instance Flowable d'un ancien ticket est liée à la Version 1 du processus. Si l'opérateur tente de faire glisser la carte vers un nouvel état qui n'existait pas en V1, le moteur rejette la transition car la flèche de transition est introuvable.

5. Solutions Architecturales & Bonnes Pratiques de Migration

✅ Stratégie A : Coexistence de Versions (Version Routing)

Approche standard en production : Les incidents déjà ouverts achèvent leur cycle de vie sous la définition V1 avec leurs colonnes d'origine. Seuls les incidents créés après la mise en production empruntent le nouveau modèle V2.

✅ Stratégie B : Migration d'Instances Active (Flowable Migration API)

Utilisation de l'API native de Flowable (`ProcessInstanceMigrationBuilder`) pour exécuter un script de migration en base de données qui mappe explicitement l'ancienne tâche `taskNouveau` vers la nouvelle tâche `taskValidation`.

Recommandation pour la Soutenance :

"Démontrer que rendre les colonnes du Kanban dynamiques est la cible idéale, mais souligner avec maturité qu'en entreprise, toute modification d'un schéma de workflow impose une politique de migration d'instances pour éviter de corrompre le cycle de vie des dossiers en cours."